

Backend Developer — Step 3

Node.js & Express.js

Roadmap objective: Build server-side applications using Node.js and Express

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Node.js runtime
- npm and package.json
- CommonJS vs ESM
- HTTP server
- Express app
- Routes and middleware
- Request/response
- Environment variables

Core concepts

1. Node.js runs JavaScript outside the browser using the V8 engine and exposes server/runtime APIs.
2. Express is a web framework for Node.js that simplifies routing, middleware and HTTP application structure.
3. Middleware can inspect/modify requests, end a response, or pass control to the next middleware.
4. Keep configuration such as secrets outside source code.

Concept map

```
Node.js & Express.js
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Node.js runtime

Study node.js runtime through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it

belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. npm and package.json

Study npm and package.json through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. CommonJS vs ESM

Study commonjs vs esm through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. HTTP server

Study http server through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Express app

Study express app through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Routes and middleware

Study routes and middleware through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Minimal Express server

```
const express = require("express");
const app = express();

app.use(express.json());

app.get("/api/health", (req, res) => {
  res.status(200).json({ status: "ok" });
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Node.js	JavaScript runtime	Runs server-side JS
Express	Web framework	Routing/middleware
npm	Package manager	Dependencies/scripts

Common mistakes

- Putting all routes in one file
- Hard-coding secrets
- Ignoring middleware order
- Returning inconsistent JSON errors
- Not handling async errors

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Create a fresh Express project from an empty folder.
- Write down what you expected, what happened, and why.

Lab 2

- Add one route and one JSON request body.
- Write down what you expected, what happened, and why.

Lab 3

- Split routes into a separate module.
- Write down what you expected, what happened, and why.

Lab 4

- Add logging middleware and observe request order.
- Write down what you expected, what happened, and why.

Lab 5

- Add a centralized error response format.
- Write down what you expected, what happened, and why.

Lab 6

- Move configuration to environment variables and test the app with a different port.
- Write down what you expected, what happened, and why.

Mini-project

Build a Node/Express Notes API with JSON requests, route modules, middleware, error handling and environment-based configuration.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

What is Node.js?

- Answer structure: definition → example → trade-off/limitation → practical use.

What does Express add?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is middleware?

- Answer structure: definition → example → trade-off/limitation → practical use.

What belongs in environment variables?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do request and response objects work?

- Answer structure: definition → example → trade-off/limitation → practical use.

How would you structure a growing Express project?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Runtime	Write this in your own words.	Give one project example.
npm	Write this in your own words.	Give one project example.
Package	Write this in your own words.	Give one project example.
Route	Write this in your own words.	Give one project example.
Middleware	Write this in your own words.	Give one project example.
Request	Write this in your own words.	Give one project example.
Response	Write this in your own words.	Give one project example.
Module	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Environment variable	Write this in your own words.	Give one project example.
Framework	Write this in your own words.	Give one project example.

Self-test

- Explain Node.js runtime without using its textbook definition.
- Show a small example of npm and package.json.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Node.js runtime.
- I can explain and demonstrate npm and package.json.
- I can explain and demonstrate CommonJS vs ESM.
- I can explain and demonstrate HTTP server.
- I can explain and demonstrate Express app.
- I can explain and demonstrate Routes and middleware.
- I can explain and demonstrate Request/response.
- I can explain and demonstrate Environment variables.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://nodejs.org/docs/latest/api/>
- <https://expressjs.com/>

Recommended videos

- Sheryians Coding School — Backend Node.js Series: Mastering Node.js Fundamentals
<https://www.youtube.com/watch?v=jhihI4kBAN8> (NEW)
- Node.js Crash Course - Traversy Media (Existing DB resource 101)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.